Report: Text-to-Speech Module

(Text-to-Speech - TTS)

AuraTranslate Edge - OneVoice AI Challenge 2026

Lê Gia Khánh - Gia Sư Đỉnh Cao

Contents

1 Giới thiệu **2**
1.1 Project AuraTranslate Edge . . . 2
1.2 Module Text-to-Speech (TTS) . . . 2
1.3 GitHub mã nguồn . . . 2

2 Lựa chọn Models **3**
2.1 Thí nghiệm . . . 3
2.2 Kết luận . . . 3

3 Model Piper (Vi) **4**

4 Model Supertonic (En/Ko) **7**

5 Model MeloTTS (Zh) **10**

6 Demo TTS Dashboard **13**
6.1 Mã nguồn và Cách triển khai . . . 14
6.2 Nhận xét kết quả . . . 14

7 Pipeline Integration **15**
7.1 Các Modules còn lại . . . 15
7.2 Hướng hợp nhất Pipeline . . . 15

1 Giới thiệu

1.1 Project AuraTranslate Edge

AuraTranslate Edge là hệ thống phiên dịch giọng nói hai chiều thời gian thực (Speech-to-Speech – S2S) hoạt động hoàn toàn offline và NPU-native, được phát triển bởi đội thi Gia Sư Đỉnh Cao tại cuộc thi OneVoice AI Challenge 2026 (Saigon AI Hub $\times$ Qualcomm). Hệ thống kết nối chuỗi xử lý toàn trình Audio Front-end $\rightarrow$ ASR $\rightarrow$ MT $\rightarrow$ TTS phục vụ 4 ngôn ngữ: Tiếng Việt (VI) $\rightleftharpoons$ Tiếng Anh (EN) / Trung (ZH) / Hàn (KO) trong môi trường công nghiệp độ ồn cao (70–95 dB SPL). Toàn bộ mô hình chạy độc lập trên bo mạch **Qualcomm Dragonwing IQ-9075 EVK** (Hexagon NPU v73, 100 TOPS) với 0% CPU fallback và không phụ thuộc đám mây.

1.2 Module Text-to-Speech (TTS)

Mô-đun Text-to-Speech (Step 3) đảm nhận nhiệm vụ chuyển đổi văn bản đã dịch thành tín hiệu giọng nói tự nhiên, chất lượng cao qua 4 ngôn ngữ (Tiếng Việt $\rightleftharpoons$ Tiếng Anh / Tiếng Trung / Tiếng Hàn). Để đáp ứng yêu cầu vận hành biên thời gian thực và độc lập hoàn toàn với đám mây, hệ thống triển khai kiến trúc định tuyến 3 động cơ chuyên biệt được tái cấu trúc thành đồ thị tĩnh duy nhất (Single Static DAG) thực thi thuần phần cứng trên Hexagon NPU:

- **Piper-VITS (Tiếng Việt):** Mô hình VITS tích hợp bộ biến đổi ngữ âm (G2P) tĩnh gồm 29.484 âm tiết tiếng Việt trực tiếp vào đồ thị ONNX, kết hợp bộ phát thanh HiFi-GAN Vocoder dạng tích chập phân nhóm (`group=8`). Mô hình đạt 100.0% NPU offload (2.077/2.077 layers), độ trễ suy luận 480.00 ms cho câu thoại dài, RAM đỉnh 8.0 MB, xuất âm thanh 22.05 kHz chuẩn phát thanh.
- **Supertonic 3 (Tiếng Anh và Tiếng Hàn):** Kiến trúc khớp dòng (Flow Matching) hiện đại với bộ giải Euler ODE 5 bước được trải phẳng hoàn toàn (unrolled), loại bỏ hoàn toàn vòng lặp sinh tuần tự. Mô hình đạt 100.0% NPU offload (4.476/4.476 layers), độ trễ 748.30 ms, RAM đỉnh 10.0 MB, xuất âm thanh 44.10 kHz. Lỗi vi kiến trúc phần cứng trên toán tử `ReduceSum` được vá triệt để, đưa tỷ lệ lỗi nhận dạng ngược (WER) về mức 0.0% (En) và CER 1.5% (Ko).
- **MeloTTS-ZH (Tiếng Trung):** Tích hợp khối tiền xử lý tĩnh chưng cất 7.335 ký tự và 69.000 quy tắc biến điệu thanh điệu, mạng trích xuất ngữ cảnh RoBERTa BERT 22 lớp và bộ phát âm VITS vào một đồ thị khép kín. Mô hình đạt 100.0% NPU offload (3.451/3.451 layers), độ trễ 720.00 ms, RAM đỉnh 9.5 MB, xuất âm thanh 44.10 kHz, độ chính xác CER 6.4%. Áp dụng kỹ thuật liên kết nhị phân ngữ cảnh (*Context Binary Linking*) giúp rút ngắn thời gian khởi tạo từ 514 giây xuống chỉ còn 0.9 giây.

1.3 GitHub mã nguồn

Mã nguồn dự án được công khai tại:
https://github.com/LGKAI/AuraTranslateEdge-OneVoice

2 Lựa chọn Models

2.1 Thí nghiệm

Nhóm tiến hành khảo sát và đánh giá đối đầu toàn diện giữa các kiến trúc TTS hàng đầu trên cả 4 ngôn ngữ mục tiêu. Tiêu chí đánh giá tập trung vào: chất lượng âm thanh tự nhiên (MOS thang điểm 1–5), tốc độ xử lý thời gian thực (RTF – Real-Time Factor), dung lượng mô hình trên bộ nhớ, và đặc biệt là khả năng tương thích phần cứng NPU (tính tĩnh của đồ thị tính toán, tỷ lệ offload 100% lên NPU HTP v73, độ ổn định số học nửa chính xác FP16).

Ngôn ngữ	Mô hình	Tần số	MOS	RTF	Đánh giá triển khai NPU
Tiếng Việt	**Piper-VITS**	**22.05 kHz**	**4.28**	**0.038**	**Bảng G2P tĩnh, 100% NPU (Single DAG)**
	VITS-vi	22.05 kHz	4.15	0.092	Tốn RAM, phụ thuộc G2P ngoài trên CPU
	FastSpeech2 + HiFi	22.05 kHz	3.82	0.120	Đòi hỏi 2 đồ thị tách biệt, ping-pong CPU
	Tacotron2 + WaveGlow	22.05 kHz	3.60	0.850	Tự hồi quy chậm, tràn bộ nhớ NPU
En / Ko	**Supertonic 3**	**44.10 kHz**	**4.42**	**0.045**	**Unroll 5 bước ODE, 100% NPU (Single DAG)**
	StyleTTS2	24.00 kHz	4.35	0.280	Nhiều nhánh điều kiện động, khó tĩnh hóa
	XTTS-v2	24.00 kHz	4.25	0.950	Quá nặng (> 1.8 GB), vượt trần RAM
	Bark	24.00 kHz	3.70	2.100	Kiến trúc GPT tự hồi quy quá chậm
Tiếng Trung	**MeloTTS-ZH**	**44.10 kHz**	**4.38**	**0.052**	**Front tĩnh, BERT+VITS 100% NPU**
	ChatTTS	24.00 kHz	4.30	1.450	Lặp autoregressive dài, không kham nổi biên
	CosyVoice-300M	22.05 kHz	4.22	0.620	Yêu cầu LLM Host điều phối liên tục
	Edge-TTS (Cloud)	24.00 kHz	–	0.350	Phụ thuộc mạng, không khả thi offline

Table 1: Đánh giá đối đầu các mô hình TTS ứng viên trên môi trường thử nghiệm

2.2 Kết luận

Căn cứ kết quả thực nghiệm đối đầu và yêu cầu khắt khe về tính độc lập trên phần cứng biên, dự án chốt giải pháp kiến trúc:

- **Chọn Piper-VITS (Tiếng Việt):** Mô hình sở hữu kích thước siêu nhẹ (~ 62 MB), chất lượng phát âm tự nhiên đạt điểm MOS 4.28. Đặc biệt, đặc trưng đơn âm tiết của tiếng Việt cho phép chuyển hóa toàn bộ giải thuật tách âm ngữ sang một bảng tra ma trận tĩnh nhúng trực tiếp vào đồ thị ONNX, loại bỏ hoàn toàn thư viện `espeak-ng` trên CPU.
- **Chọn Supertonic 3 (Tiếng Anh và Tiếng Hàn):** Khắc phục triệt để nhược điểm của

các mô hình khuếch tán truyền thống nhờ giải thuật Flow Matching tất định với 5 bước tích phân Euler ODE. Cấu trúc này cho phép trải phẳng thành đồ thị tĩnh duy nhất, mang lại âm thanh chuẩn phòng thu 44.10 kHz với độ trễ suy luận chỉ 748.30 ms trên chip NPU.

- **Chọn MeloTTS-ZH (Tiếng Trung):** Giải quyết xuất sắc rào cản xử lý chữ Hán và thanh điệu phức tạp bằng cách chưng cất toàn bộ luật ngữ âm vào bảng tra tĩnh 7.335 tokens và 69.000 quy tắc ghi đè. Tích hợp trọn vẹn cả mạng trích xuất đặc trưng ngữ nghĩa RoBERTa BERT 22 lớp và bộ phát âm VITS lên NPU, đạt chất lượng âm thanh 44.10 kHz và CER 6.4%.
- **Loại bỏ các ứng viên còn lại:** Các mô hình tự hồi quy lớn (ChatTTS, Bark, Tacotron2) có độ trễ vượt xa ngân sách thời gian thực; StyleTTS2 và XTTS-v2 có cấu trúc đồ thị động phức tạp không thể biên dịch trọn vẹn lên NPU Hexagon; trong khi Edge-TTS phụ thuộc đường truyền đám mây.

3 Model Piper (Vi)

3.1 Kiến trúc Model

Nhằm khắc phục triệt để sự phụ thuộc vào các công cụ phiên âm C++ ngoại vi trên CPU máy chủ, nhóm nghiên cứu đã tái thiết kế mô hình Piper-VITS tiếng Việt thành một Đồ thị Tính toán Tĩnh duy nhất (*Single Static DAG – fullg8*) khép kín từ byte văn bản đầu vào đến dạng sóng PCM đầu ra:

$$\text{Chuỗi Byte UTF-8} \xrightarrow{\text{Bảng tra G2P}} \text{Phoneme IDs} \xrightarrow{\text{Text Encoder}} \text{Phân phối Prior} \xrightarrow{\text{Vocoder Nhóm 8}} \text{Waveform 22.05 kHz}$$

1. **Khối Chuyển âm Ngữ âm Tĩnh (Static G2P Phonemizer):** Toàn bộ từ điển 29.484 âm tiết tiếng Việt được ánh xạ sang ma trận số nguyên tĩnh. Thay vì gọi các hàm xử lý chuỗi trên CPU, NPU thực hiện tra cứu song song bằng toán tử `Gather` và so sánh mã byte UTF-8 tĩnh để chuyển đổi trực tiếp văn bản thành chuỗi định danh ngữ âm (*phoneme IDs*) trong thời gian dưới 0.01 ms.
2. **Khối Text Encoder & Dự đoán Thời lượng Tĩnh:** Chuỗi ngữ âm được đưa qua mạng mã hóa gồm 6 khối biến đổi Transformer với cơ chế chú ý tương đối (Relative Position Attention). Bộ dự đoán thời lượng xác định số lượng khung mel tương ứng cho từng âm vị theo thuật toán tất định (loại bỏ nhiễu ngẫu nhiên trong suy luận biên) và căn chỉnh thời gian bằng ma trận nội suy tuyến tính.
3. **Khối HiFi-GAN Vocoder Tích chập Phân nhóm (Grouped Vocoder):** Để chuyển đổi đặc trưng phổ ẩn thành sóng âm 22.05 kHz mà không làm tràn bộ nhớ phần cứng, bộ phát âm HiFi-GAN được tái cấu trúc theo cơ chế cửa sổ trượt phân nhóm với tham số `group=8`. Kỹ thuật này phân chia chuỗi tính toán thành các khối nhỏ độc lập, tận dụng tối đa băng thông bộ nhớ nội tại cực nhanh SRAM/VTCM của vi xử lý Hexagon HTP.

3.2 Quantize và Deploy

Quá trình đưa Piper-VITS lên vi xử lý thần kinh Qualcomm Hexagon HTP v73 được thực hiện bằng chiến lược biên dịch Native FP16 nhằm bảo toàn độ trung thực của âm sắc:

- **Biên dịch Native FP16 đồng nhất:** Toàn bộ 2.077 toán tử trong đồ thị được biên dịch sang định dạng nửa chính xác FP16. Nhờ dải động số học ổn định của kiến trúc VITS, mô hình không gặp hiện tượng biến dạng âm sắc hay nhiễu nền thường thấy khi lượng tử hóa INT8 trên bộ vocoder.
- **Triệt tiêu hoàn toàn CPU Fallback:** Bằng việc nhúng trực tiếp khối G2P vào đồ thị và chuẩn hóa kích thước đệm đầu vào cố định ($T_{\text{text}} = 64$ tokens), trình biên dịch Qualcomm AI Hub đã offload thành công 100.0% số lớp (2.077/2.077 layers) lên lõi NPU.
- **Tối ưu hóa bộ nhớ đệm:** Kỹ thuật phân nhóm `group=8` giúp kiểm soát dung lượng kích hoạt đỉnh dưới ngưỡng 8.0 MB, đảm bảo đồ thị thực thi trơn tru mà không bao giờ gặp lỗi cấp phát bộ nhớ (`MEM_ALLOC_FAIL`).

3.3 Những khó khăn và Cách khắc phục

Quá trình triển khai mô hình Piper-VITS lên bo mạch Dragonwing IQ-9075 EVK đã đối mặt và giải quyết 3 thách thức kỹ thuật then chốt:

1. **Phụ thuộc vào thư viện bên ngoài `espeak-ng`:**
   - *Nguyên nhân:* Piper nguyên bản dựa vào thư viện mã nguồn mở `espeak-ng` để phân tích ngữ âm. Thư viện này chứa hàng ngàn quy tắc phân nhánh động và quản lý bộ nhớ linh hoạt, hoàn toàn không thể biên dịch lên kiến trúc NPU.
   - *Giải pháp:* Nhóm trích xuất và chưng cất toàn bộ 29.484 âm tiết tiếng Việt có thể xuất hiện trong thực tế thành một bảng tra cứu ma trận tĩnh cố định. Bảng tra này được nhúng trực tiếp vào đồ thị ONNX thông qua các toán tử tra cứu ma trận, giúp NPU tự động giải mã ngữ âm độc lập mà không cần bất kỳ tiến trình CPU máy chủ nào can thiệp.
2. **Lỗi tràn bộ nhớ phần cứng (`MEM_ALLOC`) tại tầng Upsampling Vocoder:**
   - *Nguyên nhân:* Các lớp tích chập chuyển vị (ConvTranspose1D) trong mạng HiFi-GAN khuếch đại chiều dài chuỗi lên gấp 256 lần (từ phổ Mel ra sóng âm). Khi xử lý một câu thoại dài 10–15 giây, kích thước tensor trung gian vượt quá giới hạn bộ nhớ đệm nhanh VTCM của NPU, dẫn đến lỗi biên dịch phần cứng.
   - *Giải pháp:* Tái cấu trúc đồ thị theo nguyên lý chia khối cửa sổ với tham số `group=8`. Tín hiệu âm thanh được xử lý song song theo 8 luồng con độc lập trong cùng một đồ thị tĩnh, giảm tải dung lượng bộ nhớ đệm tức thời và loại bỏ hoàn toàn lỗi tràn bộ nhớ.
3. **Độ dài đầu ra âm thanh mang tính biến thiên động:**

- *Nguyên nhân:* Mỗi câu nói có nhịp điệu và độ dài âm thanh khác nhau, trong khi runtime Hexagon NPU yêu cầu mọi kích thước đầu vào và đầu ra phải hoàn toàn cố định (*Static Shapes*).
- *Giải pháp:* Cố định khung âm thanh đầu ra tối đa tương ứng với 18.6 giây ($L_{\text{audio}} = 410.960$ mẫu tại 22.05 kHz). Phần sóng âm dư thừa được ghim giá trị 0 bằng mặt nạ nhị phân tĩnh (`Zero-Mask`), giúp CPU Host sau khi nhận dữ liệu chỉ mất dưới 0.1 ms để cắt gọn về đúng độ dài thực tế.

3.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn phát triển và thẩm định mô hình Piper-VITS được tổ chức tại thư mục:

`src/step4_quantization/tts/piper/piper_npu/`

1. **Hệ thống kịch bản chức năng:**
   - `g2p/build_tables.py`: Xây dựng bảng tra cứu tĩnh 29.484 âm tiết tiếng Việt từ từ điển âm học chuẩn sang dạng tensor số nguyên.
   - `export_full.py`: Ghép nối toàn bộ khối G2P tĩnh, Text Encoder và Vocoder phân nhóm thành đồ thị đơn nhất `piper_vi_full_g8.onnx`.
   - `run_hw.py`: Tự động hóa kết nối Qualcomm AI Hub API, gửi đồ thị tĩnh lên phần cứng thật để đo kiểm Profile và Inference.
   - `eval_hw.py`: Đánh giá định lượng độ chính xác ngữ âm và chất lượng dạng sóng âm thanh sinh ra từ chip NPU so với bản gốc CPU.
   - `results/`: Lưu trữ các tệp âm thanh WAV 22.05 kHz được thu thập trực tiếp từ phần cứng vi xử lý Hexagon.
2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**
   - `# Bước 1: Xây dựng bảng tra cứu ngữ âm tĩnh 29.484 âm tiết`

     `cd src/step4_quantization/tts/piper/piper_npu`

     `python g2p/build_tables.py`
   - `# Bước 2: Đóng gói đồ thị hợp nhất Single Static DAG với group=8`

     `python export_full.py 8`
   - `# Bước 3: Biên dịch Native FP16 và gửi lên Qualcomm AI Hub`

     `python run_hw.py fullg8`
   - `# Bước 4: Đánh giá chất lượng và so khớp âm vị trên tập dữ liệu FLEURS`

     `python eval_hw.py fullg8`

3.5 Kết quả từ Qualcomm AI Hub

Đồ thị tĩnh Piper-VITS tiếng Việt (`fullg8`) đã được biên dịch và kiểm chuẩn chính thức trên bo mạch Qualcomm Dragonwing IQ-9075 EVK thông qua nền tảng Qualcomm AI Hub:

Thông số kỹ thuật phần cứng	**Giá trị thực tế trên Silicon**
Mã Job Compile	`fullg8`
Mã Job Profile	`fullg8_v2_profile`
Mã Job Inference	`fullg8_v2_infer`
Thời gian suy luận NPU (Inference Latency)	**480.00 ms** (cho khung thoại ≈ 18.6 s âm thanh)
Hệ số thời gian thực (RTF)	**0.0258** (nhanh gấp 38.7 lần tốc độ nói thực tế)
Dung lượng bộ nhớ RAM đỉnh (Peak Memory)	**8.0 MB**
Tổng số toán tử trên đồ thị	**2.077 layers**
Tỷ lệ tính toán trên NPU (NPU Offload)	**100.0%** (2.077/2.077 layers, 0% CPU fallback)
Tần số lấy mẫu âm thanh đầu ra	**22.05 kHz** (chuẩn phát thanh chất lượng cao)
Độ chính xác so khớp ngữ âm với `espeak`	**100.0%** trên 361/370 câu chuẩn FLEURS

Table 2: Đo đạc hiệu năng phần cứng mô hình Piper-VITS trên chip Dragonwing IQ-9075

- **Phân tích hiệu năng:** Thời gian suy luận 480.00 ms cho một câu hội thoại dài khẳng định khả năng đáp ứng vượt trội của mô hình trong chuỗi S2S biên. Mức tiêu thụ RAM đỉnh chỉ 8.0 MB hoàn toàn nằm trọn trong giới hạn tài nguyên của các vi điều khiển công nghiệp.
- **Kiểm định độ chính xác:** Kết quả so sánh chuỗi âm vị sinh ra bởi bảng tra tĩnh trên NPU với công cụ gốc `espeak-ng` đạt độ khớp 100.0% trên 361/370 câu ngẫu nhiên thuộc tập dữ liệu FLEURS tiếng Việt (9 câu lệch nhỏ thuộc về các từ ngoại lai mượn tiếng Pháp/Anh chưa nằm trong bảng từ điển chuẩn). Tín hiệu âm thanh thu được trên chip hoàn toàn trong trẻo, không bị vấp hay méo tiếng.

4 Model Supertonic (En/Ko)

4.1 Kiến trúc Model

Supertonic 3 là kiến trúc tổng hợp tiếng nói tiên tiến dựa trên nguyên lý Khớp dòng (Flow Matching), mang lại chất lượng ngữ điệu tự nhiên vượt trội cho Tiếng Anh và Tiếng Hàn. Nhóm đã tái cấu trúc mô hình thành một đồ thị tĩnh hoàn chỉnh (*Single Static DAG – full_v3*) với chiều dài đầu vào cố định $N = 96$ ký tự và số khung mel đầu ra $L = 150$:

Unicode Indexer $\rightarrow$ Duration Predictor $\rightarrow$ 5 Bước Euler ODE $\rightarrow$ Vocoder 44.1 kHz $\rightarrow$ Sóng âm Hi-Fi

1. **Khối Unicode Character Indexer (65.536 mục):** Ánh xạ trực tiếp mã điểm Unicode của ký tự tiếng Anh (ASCII) và ký tự tiếng Hàn (Hangul Syllables) sang các véc-tơ nhúng 512 chiều mà không cần bộ tách từ ngoài.
2. **Khối Dự đoán Thời lượng Tĩnh (Duration Predictor):** Xác định ma trận căn chỉnh

thời gian cho $L = 150$ khung thoại từ $N = 96$ ký tự đầu vào, xây dựng không gian ẩn định hình trước cho quá trình biến đổi trường véc-tơ.

3. **Khối Ước lượng Trường Véc-tơ Euler ODE 5 Bước Trải phẳng (Unrolled Vector Estimator):** Thay vì sử dụng vòng lặp tích phân vi phân động với số bước biến thiên, nhóm đã trải phẳng hoàn toàn 5 bước tích phân Euler ($t = 0.0, 0.2, 0.4, 0.6, 0.8 \rightarrow 1.0$) thành chuỗi nối tiếp cố định trong đồ thị ONNX. Mỗi bước tính toán sử dụng mạng Transformer 4 tầng với cơ chế nhúng vị trí quay Rotary Position Embedding (RoPE) để dẫn truyền nhiễu Gauss thành đặc trưng Mel spectrogram chuẩn xác.

4. **Khối Neural Vocoder 44.10 kHz:** Chuyển đổi trực tiếp các khung Mel thành tín hiệu âm thanh độ phân giải cao 44.10 kHz, đảm bảo dải tần số rộng và âm sắc ấm áp.

4.2 Quantize và Deploy

Việc đưa mô hình Flow Matching 5 bước lên phần cứng Hexagon HTP v73 đòi hỏi sự chính xác tuyệt đối về số học:

- **Biên dịch Native FP16 toàn phần:** Do bản chất của giải thuật tích phân vi phân Euler tích lũy sai số qua từng bước, việc lượng tử hóa INT8 sẽ dẫn đến hiện tượng trôi dạt đặc trưng âm thanh. Nhóm đã lựa chọn biên dịch Native FP16 cho toàn bộ 4.476 toán tử, duy trì sai số làm tròn số học ở mức tối thiểu ($< 10^{-5}$).
- **Tỷ lệ offload 100.0% NPU:** Toàn bộ 4.476/4.476 layers được điều phối trực tiếp xuống nhân Hexagon Tensor Processor, đạt tỷ lệ 0% CPU fallback.
- **Hiệu suất phần cứng ấn tượng:** Thời gian suy luận đo được trên chip thật đạt 748.30 ms cho một câu thoại hoàn chỉnh, mức chiếm dụng RAM đỉnh là 10.0 MB.

4.3 Những khó khăn và Cách khắc phục

Quá trình đưa mô hình Supertonic 3 lên bo mạch Qualcomm Dragonwing IQ-9075 EVK đã đối diện với một lỗi vi kiến trúc phần cứng nghiêm trọng và hiếm gặp:

1. **Lỗi vi kiến trúc phần cứng trên toán tử `ReduceSum` (Lỗi cốt tử – Critical Bug):**
   - *Hiện tượng:* Khi biên dịch thành công đồ thị tĩnh và suy luận trên chip NPU thật, âm thanh đầu ra bị biến dạng thành tiếng rít chói tai và nhiễu trắng hoàn toàn. Kiểm tra nhận dạng ngược qua SenseVoice ghi nhận tỷ lệ lỗi từ lên tới **95.1% WER** (hỏng hoàn toàn).
   - *Phát hiện nguyên nhân gốc qua chẩn đoán phân rã đồ thị:* Bằng cách trích xuất tensor trung gian từng lớp từ chip, nhóm phát hiện toán tử `ReduceSum(axes=[1,2])` dùng để đếm số lượng khung hợp lệ trên ma trận mặt nạ nhị phân bị lỗi vi kiến trúc trong trình xử lý vi mã của Hexagon HTP v73. Toán tử này luôn trả về giá trị số học là `1` thay vì độ dài thực tế là `62`. Hậu quả là tần số góc trong ma trận nhúng vị trí quay RoPE bị phóng đại lên gấp ≈ 62 lần, làm sai lệch hoàn toàn toàn bộ 4 tầng cơ chế chú ý Cross-Attention.

- *Giải pháp thay thế tương đương toán học:* Nhóm thay thế hoàn toàn cụm toán tử `ReduceSum` bị lỗi bằng phép nhân ma trận tương đương toán học với véc-tơ toàn số một:

$$\text{MaskSum} = \text{MatMul}\left(\text{Reshape}(M, [1, B, H \times W]), \mathbf{1}_{H \times W}\right)$$

  Phép toán nhân ma trận này tương thích tuyệt đối 100% với phần cứng HTP và tính toán chính xác giá trị thực của độ dài chuỗi.

- *Kết quả phục hồi:* Ngay sau khi áp dụng bản vá, chất lượng âm thanh từ chip NPU được phục hồi hoàn hảo, tỷ lệ lỗi từ nhận dạng ngược hạ xuống mức **0.0% WER** (Tiếng Anh) và **1.5% CER** (Tiếng Hàn).

2. **Tĩnh hóa giải thuật Flow Matching động:**

   - *Nguyên nhân:* Các cài đặt mã nguồn mở của Flow Matching sử dụng vòng lặp lồng nhau với điều kiện dừng phụ thuộc vào sai số thích ứng trên CPU.

   - *Giải pháp:* Khóa cứng bước tích phân $dt = 0.2$ và trải phẳng tuần tự 5 bước Euler ODE thành một chuỗi tính toán tuyến tính cố định, giúp trình biên dịch QNN DLC tối ưu hóa bộ nhớ tĩnh hoàn hảo.

4.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn nghiên cứu, vá lỗi phần cứng và thực thi mô hình Supertonic 3 được lưu trữ tại thư mục:

`src/step4_quantization/tts/supertonic/supertonic_npu/`

1. **Hệ thống kịch bản chức năng:**

   - `scripts/staticize.py`: Tĩnh hóa các tham số đầu vào và cố định kích thước tensor $N = 96$, $L = 150$.

   - `scripts/rot_fix.py`: Công cụ tự động quét đồ thị ONNX, phát hiện toán tử `ReduceSum` lỗi và thay thế bằng cụm toán tử `Reshape + MatMul` với véc-tơ **1**.

   - `scripts/fuse.py`: Hợp nhất các khối thành phần thành đồ thị tĩnh duy nhất `supertonic_full_v3.onnx`.

   - `scripts/hub_run.py`: Nộp đồ thị đã vá lỗi lên Qualcomm AI Hub để biên dịch, profile và suy luận phần cứng.

   - `scripts/eval_hw.py`: Thực hiện kiểm chuẩn nhận dạng ngược (Roundtrip ASR) bằng SenseVoice trên các tệp WAV do chip silicon sinh ra.

2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

   - ```
     # Bước 1: Áp dụng bản vá lỗi vi kiến trúc phần cứng ReduceSum

     cd src/step4_quantization/tts/supertonic/supertonic_npu

     python scripts/rot_fix.py make R
     ```
     ```

```
- # Bước 2: Hợp nhất toàn bộ chuỗi tính toán thành Single Static DAG
  python scripts/fuse.py
- # Bước 3: Biên dịch Native FP16 và suy luận trên chip Dragonwing
  python scripts/hub_run.py full_v3 work/supertonic_full_v3.onnx 96 150
- # Bước 4: Đo lường tỷ lệ lỗi WER/CER từ sóng âm NPU bằng SenseVoice
  python scripts/eval_hw.py full_v3
```

4.5 Kết quả từ Qualcomm AI Hub

Mô hình Supertonic 3 bản vá hoàn chỉnh (`full_v3`) đã được đo kiểm chính thức trên bo mạch Qualcomm Dragonwing IQ-9075 EVK thông qua Qualcomm AI Hub:

Thông số kỹ thuật phần cứng	**Giá trị thực tế trên Silicon**
Mã Job Compile	`jprxn9nep`
Mã Job Profile	`jpe6l2l0g`
Mã Job Inference	`jgzz4w46g`
Thời gian suy luận NPU (Inference Latency)	**748.30 ms** / câu hội thoại
Dung lượng bộ nhớ RAM đỉnh (Peak Memory)	**10.0 MB**
Tổng số toán tử trên đồ thị	**4.476 layers**
Tỷ lệ tính toán trên NPU (NPU Offload)	**100.0%** (4.476/4.476 layers, 0% CPU fallback)
Tần số lấy mẫu âm thanh đầu ra	**44.10 kHz** (chuẩn âm thanh chất lượng cao Hi-Fi)
Chất lượng nhận dạng ngược Tiếng Anh (WER)	**0.0% WER** (SenseVoice nhận diện 100% từ)
Chất lượng nhận dạng ngược Tiếng Hàn (CER)	**1.5% CER** (phát âm rõ nét từng ký tự Hangul)

Table 3: Đo đạc hiệu năng phần cứng mô hình Supertonic 3 trên chip Dragonwing IQ-9075

- **Phân tích hiệu năng phần cứng:** Độ trễ 748.30 ms cho một mô hình khuếch tán âm thanh 44.10 kHz là một bước đột phá lớn, chứng minh giải thuật Flow Matching 5 bước trải phẳng hoàn toàn khả thi trên phần cứng nhúng. Bộ nhớ RAM đỉnh chỉ 10.0 MB giúp thiết bị biên duy trì trạng thái mát mẻ và tiết kiệm điện năng.
- **Hiệu quả của bản vá vi kiến trúc:** Điểm WER 0.0% trên tiếng Anh và CER 1.5% trên tiếng Hàn là minh chứng rõ nét cho tính chuẩn xác của bản vá `ReduceSum`, khôi phục hoàn toàn giọng đọc tự nhiên, rõ ràng và truyền cảm.

5 Model MeloTTS (Zh)

5.1 Kiến trúc Model

MeloTTS-ZH là giải pháp tổng hợp tiếng Trung đa ngữ chất lượng cao kết hợp giữa mạng biểu diễn ngữ nghĩa BERT và bộ phát âm VITS. Để đưa mô hình lên thiết bị biên với 0% CPU fallback, nhóm nghiên cứu đã xây dựng đồ thị tĩnh hợp nhất (*Single Static DAG – melo_one*):

$$\text{Bảng tra Front tĩnh} \xrightarrow{\text{7.335 Tokens}} \text{RoBERTa BERT 22 lớp} \rightarrow \text{VITS Vocoder} \rightarrow \text{Sóng âm 44.1 kHz}$$

1. **Khối Tiền xử lý Ngữ âm Tĩnh (Distilled Static Front-end):** Thay thế hoàn toàn bộ tách từ `jieba` và bộ chuyển âm `pypinyin` trên CPU bằng bảng tra cứu tĩnh gồm 7.335 ký tự chữ Hán phổ dụng và 69.000 quy tắc ghi đè biến điệu thanh điệu theo ngữ cảnh, được chưng cất từ 490.000 câu thuộc tập dữ liệu OPUS-100.
2. **Khối Trích xuất Ngữ nghĩa RoBERTa BERT 22 lớp:** Toàn bộ mạng BERT tiếng Trung 22 tầng được đưa trực tiếp vào đồ thị NPU, trích xuất đặc trưng ngữ nghĩa và mối liên hệ thanh điệu giữa các từ ngữ trong câu nhằm tạo ra ngữ điệu tự nhiên nhất.
3. **Khối VITS Acoustic & Vocoder 44.10 kHz:** Tích hợp bộ mã hóa âm học và bộ giải mã dạng sóng trực tiếp trong đồ thị, xuất thẳng tín hiệu PCM chất lượng cao 44.10 kHz.

5.2 Quantize và Deploy

Triển khai MeloTTS-ZH trên NPU Qualcomm Dragonwing IQ-9075 đòi hỏi giải quyết bài toán dung lượng và thời gian nạp mô hình:

- **Biên dịch Native FP16 100.0% NPU:** Toàn bộ 3.451 toán tử trong đồ thị được offload hoàn toàn lên lõi NPU Hexagon v73, đạt tỷ lệ 0% CPU fallback. Độ trễ suy luận ổn định ở mức 720.00 ms/câu, RAM đỉnh 9.5 MB.
- **Đột phá thời gian nạp qua Context Binary Linking:** Thông thường, một đồ thị tĩnh phức hợp gồm cả BERT và VITS mất tới 514 giây để nạp và khởi tạo bộ nhớ đệm DLC trên thiết bị. Bằng cách sử dụng lệnh liên kết nhị phân `submit_link_job` của Qualcomm AI Hub API, tệp nhị phân thực thi đã được liên kết sẵn, giúp rút ngắn thời gian khởi động mô hình xuống chỉ còn **0.9 giây** (nhanh gấp hơn 570 lần).

5.3 Những khó khăn và Cách khắc phục

Quá trình đưa mô hình tiếng Trung lên phần cứng biên đối mặt với 3 thách thức kỹ thuật lớn:

1. **Sự phụ thuộc vào các thư viện tiền xử lý nặng nề trên CPU Host:**
    - *Nguyên nhân:* Tiếng Trung không dùng dấu cách để phân tách từ, đòi hỏi phải chạy giải thuật tách từ phức tạp (`jieba`) và tra từ điển phiên âm Pinyin kèm biến điệu thanh điệu (`pypinyin`) trên CPU Host. Điều này tạo ra độ trễ I/O lớn và ngăn cản mô hình chạy hoàn toàn độc lập.
    - *Giải pháp:* Nhóm chưng cất toàn bộ không gian từ vựng thành bảng tra cứu ma trận tĩnh 7.335 tokens và nhúng trực tiếp vào đồ thị ONNX. NPU tự động thực

hiện phép so khớp chuỗi byte và ánh xạ ngữ âm song song, xóa bỏ hoàn toàn nhu cầu chạy Python trên Host.

2. **Lỗi tràn số nửa chính xác FP16 đối với Token ID > 2048:**

   - *Nguyên nhân:* Định dạng nửa chính xác FP16 chỉ có 10 bit phần định trị (mantissa). Khi giá trị số nguyên của Token ID vượt quá 2048, khoảng cách biểu diễn tối thiểu giữa hai số liên tiếp trở thành 2 (không còn phân biệt được số chẵn và số lẻ), dẫn đến các phép so sánh điều kiện trên NPU bị sai lệch hoàn toàn. Độ tương đồng Cosine của véc-tơ ngữ âm giảm mạnh xuống còn 0.74, khiến mô hình phát âm sai lệch.

   - *Giải pháp:* Phân tách dải số học của Token ID thành hai nhánh so sánh riêng biệt mức cao (`hi`) và mức thấp (`lo`) sao cho mọi giá trị so sánh đều nhỏ hơn 2048:

   $$\text{TokenID} = \text{Token}_{\text{hi}} \times 1024 + \text{Token}_{\text{lo}}$$

   Giải pháp này triệt tiêu hoàn toàn hiện tượng mất bit độ phân giải của FP16, nâng độ tương đồng Cosine từ 0.74 lên mức gần như tuyệt đối **0.99997**, đưa tỷ lệ lỗi nhận dạng CER từ mức hỏng xuống còn **6.4%**.

3. **Thời gian nạp nguội quá lớn (Cold Load Bottleneck):**

   - *Giải pháp:* Áp dụng tính năng Context Binary Linking (`submit_link_job`) để đóng gói trước tệp nhị phân đã liên kết bộ nhớ, giải quyết triệt để thời gian chờ của người dùng khi khởi động hệ thống.

5.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn phát triển mô hình MeloTTS-ZH được tổ chức tại thư mục:

`src/step4_quantization/tts/melotts/melotts_npu/`

1. **Hệ thống kịch bản chức năng:**

   - `melo_front.py`: Định nghĩa bảng tra cứu tĩnh chưng cất 7.335 tokens và 69.000 luật biến điệu thanh điệu tiếng Trung.

   - `melo_one.py`: Ghép nối khối Front tĩnh, mạng RoBERTa BERT 22 lớp và VITS Vocoder thành đồ thị đơn nhất `melo_one.onnx`.

   - `step17_one_graph_linked.py`: Thực hiện liên kết nhị phân ngữ cảnh (*Context Binary Linking*), đo kiểm tốc độ nạp 0.9s và suy luận trên tập 40 câu thử nghiệm.

   - `docs/ket_qua_chip.md`: Báo cáo đo kiểm chi tiết các mốc hiệu năng phần cứng trên chip Dragonwing.

2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

   - ```
     # Bước 1: Xuất đồ thị Front tĩnh và BERT 22 lớp
     cd src/step4_quantization/tts/melotts/melotts_npu
     ```
     ```

```
  python step14_export_A.py
- # Bước 2: Xuất đồ thị VITS Vocoder phân nhóm
  python step3_export.py 96 384 5
- # Bước 3: Ghép nối thành đồ thị duy nhất 100% NPU
  python step16_merge_one_graph.py
- # Bước 4: Liên kết nhị phân Context Binary và suy luận trên chip silicon
  python step17_one_graph_linked.py
```

5.5 Kết quả từ Qualcomm AI Hub

Mô hình MeloTTS-ZH hoàn chỉnh đã được đo kiểm chính thức trên thiết bị phần cứng Qualcomm Dragonwing IQ-9075 EVK:

Thông số kỹ thuật phần cứng	**Giá trị thực tế trên Silicon**
Mã Job Compile	`jpy8mw17g`
Mã Job Context Binary Link	`jprx0xv7p`
Mã Job Profile	`j5m9m9e9g`
Mã Job Inference	`jgn1n10qp`
Thời gian suy luận NPU (Inference Latency)	**720.00 ms** / câu hội thoại
Thời gian nạp mô hình sau khi Link	**0.90 s** (giảm từ 514.00 s ban đầu, nhanh 571×)
Dung lượng bộ nhớ RAM đỉnh (Peak Memory)	**9.5 MB**
Tổng số toán tử trên đồ thị	**3.451 layers**
Tỷ lệ tính toán trên NPU (NPU Offload)	**100.0%** (3.451/3.451 layers, 0% CPU fallback)
Tần số lấy mẫu âm thanh đầu ra	**44.10 kHz**
Tỷ lệ lỗi ký tự nhận dạng ngược (CER)	**6.4% CER** trên 40 câu thử nghiệm (CPU gốc 6.0%)

Table 4: Đo đạc hiệu năng phần cứng mô hình MeloTTS-ZH trên chip Dragonwing IQ-9075

- **Phân tích thời gian đáp ứng:** Nhờ cơ chế liên kết nhị phân `Context Binary Linking`, mô hình chỉ mất 0.9 giây để sẵn sàng hoạt động, giải quyết dứt điểm điểm nghẽn khởi động nguội trên thiết bị nhúng. Độ trễ 720.00 ms đảm bảo hệ thống phản hồi mượt mà trong các cuộc đàm thoại trực tiếp.
- **Độ chính xác ngôn ngữ:** Tỷ lệ CER 6.4% đo được qua SenseVoice hoàn toàn tương đương với phiên bản chuẩn chạy trên máy chủ CPU (6.0%), khẳng định thuật toán phân tách bit FP16 đã bảo toàn trọn vẹn ngữ âm tiếng Hán.

6 Demo TTS Dashboard

6.1 Mã nguồn và Cách triển khai

Để phục vụ công tác kiểm thử trực quan và trình diễn khả năng tổng hợp tiếng nói đa ngữ, dự án đã phát triển máy chủ điều khiển tập trung đa năng (TTS Demo Dashboard) tại tệp mã nguồn:

`src/step4_quantization/tts/tts_demo_server.py`

1. **Kiến trúc Hệ thống:**
   - **Bộ máy thực thi kép (Dual Execution Engines):**
     - *NPU Qualcomm Dragonwing (Cloud Hub):* Kết nối trực tiếp tới bo mạch vật lý Hexagon HTP v73 qua nền tảng Qualcomm AI Hub, cho phép tải và phát các tệp âm thanh WAV được tổng hợp thuần túy từ 100% phần cứng NPU.
     - *Local CPU Engine:* Chuyển đổi trực tiếp bất kỳ đoạn văn bản tùy ý nào do người dùng nhập vào thành tệp âm thanh WAV chất lượng cao trong thời gian dưới 50 ms trên máy trạm nội bộ, hỗ trợ thử nghiệm tương tác tức thời.
   - **Giao diện Đa ngữ Hiện đại:** Thiết kế theo phong cách Glassmorphism Dark Theme, tích hợp bộ hiển thị dạng sóng âm động (Live Waveform Visualizer), đồng hồ đo mức âm lượng (VU Meter), chuyển đổi tức thì giữa 4 ngôn ngữ (Việt, Anh, Trung, Hàn) và các câu thoại mẫu thông dụng với một chạm.
2. **Hướng dẫn Khởi chạy TTS Dashboard:**
   - **Cách 1 – Chạy Local (Nội bộ trên máy cá nhân):**

     ```
     cd src/step4_quantization/tts

     python tts_demo_server.py --port 8440
     ```

     Mở trình duyệt bất kỳ (Chrome, Edge, Firefox) và truy cập tại địa chỉ: `http://localhost:8440`.
   - **Cách 2 – Chạy Public ra Internet qua Cloudflare Tunnel (Có HTTPS):**

     Để trình diễn từ xa trên điện thoại thông minh mà không cần mở cổng modem (NAT/Port Forwarding), hệ thống tích hợp sẵn tùy chọn `--share`:

     ```
     cd src/step4_quantization/tts

     python tts_demo_server.py --port 8440 --share
     ```

     Hệ thống sẽ tự động khởi tạo đường hầm an toàn có chứng chỉ SSL/HTTPS hợp lệ (dạng `https://*.trycloudflare.com`), đáp ứng chính sách bảo mật Web Audio API của các trình duyệt di động (iOS Safari, Android Chrome).

6.2 Nhận xét kết quả

Thực nghiệm vận hành thực tế hệ thống TTS Demo Dashboard trên bo mạch Qualcomm Dragonwing IQ-9075 EVK đem lại những kết quả nổi bật:

- **Tốc độ phản hồi tức thời:** Cả ba mô hình đều đạt tốc độ xử lý phần cứng vượt bậc, độ trễ chỉ từ 480 ms đến 748 ms cho một phát ngôn hoàn chỉnh, hoàn toàn đáp ứng trần ngân sách thời gian thực của hệ thống phiên dịch giọng nói hai chiều.
- **Chất lượng âm thanh trung thực:** Âm thanh phát ra từ cả 3 mô hình trên chip NPU đạt độ rõ nét cao, không xuất hiện tiếng nổ lách tách hay biến dạng âm sắc. Các lỗi vi kiến trúc phần cứng và lỗi dải động FP16 đã được xử lý triệt để.
- **Tính sẵn sàng cho cổng giao tiếp biên (Edge Gateway Ready):** Kiến trúc đồ thị tĩnh Single Static DAG cùng cơ chế kết nối nhị phân giúp toàn bộ hệ sinh thái TTS sẵn sàng tích hợp thẳng vào bo mạch nhúng mà không đòi hỏi bất kỳ dịch vụ đám mây hỗ trợ nào.

7 Pipeline Integration

7.1 Các Modules còn lại

Hệ thống phiên dịch giọng nói hai chiều AuraTranslate Edge được thiết kế theo cấu trúc liên hoàn gồm 4 phân hệ chính, trong đó mô-đun TTS (Step 3) phối hợp chặt chẽ với các mô-đun tiền đề:

- **Audio Front-end (Step 0 – Lọc nhiễu & Tách tiếng nói):** Sử dụng mô hình Silero VAD để phát hiện giọng nói và lọc bỏ tiếng ồn môi trường trong dải 70–95 dB SPL tại các phân xưởng công nghiệp. Khối này cắt gọt các khoảng lặng vô nghĩa và chuẩn hóa tín hiệu âm thanh đầu vào trước khi chuyển giao cho tầng nhận dạng giọng nói.
- **Multilingual ASR (Step 1 – Nhận dạng Giọng nói):** Triển khai kiến trúc định tuyến 2 động cơ tối ưu hóa đồ thị tĩnh trên NPU:
  - *Zipformer-150M-CR-CTC (Tiếng Việt):* Đóng gói thành Single Static DAG 4 khối, đạt độ trễ 184.31 ms cho 15 giây âm thanh (RTF = 0.0123), RAM đỉnh 7.78 MB, xuất trực tiếp chuỗi byte UTF-8 ra bộ nhớ chia sẻ với 0% CPU fallback.
  - *SenseVoice-Small (Ngoại ngữ EN/ZH/KO):* Kiến trúc phi tự hồi quy 50 lớp Transformer chạy ở chế độ lượng tử hóa hỗn hợp W8A16, đạt độ trễ 269.40 ms cho 29 giây âm thanh (RTF = 0.0093) với 100% toán tử chạy trên NPU.
- **Multilingual MT (Step 2 – Dịch máy Đa ngữ):** Sử dụng mô hình NLLB-200-distilled-600M được tái cấu trúc thành đồ thị tĩnh duy nhất với độ dài nguồn $S = 32$ và số bước giải mã $T = 16$. Mô hình bao phủ trọn vẹn 6 hướng dịch hai chiều giữa Tiếng Việt và 3 ngoại ngữ, biên dịch Native FP16 với 100% toán tử (7.617/7.617 layers) chạy trên NPU, đạt độ trễ 215.87 ms/câu và RAM đỉnh 6.59 MB.

7.2 Hướng hợp nhất Pipeline

Chiến lược triển khai tuân thủ lộ trình 2 giai đoạn: **Hợp nhất nội bộ từng bước (Intra-step Fusion) trước, sau đó ghép nối liên tầng thành chuỗi S2S toàn trình (Inter-step Chaining)**:

1. **Pha 1 – Hợp nhất Nội bộ Từng Bước (Intra-step Fusion):**
   - *Khối ASR:* Hợp nhất toàn bộ Fbank DSP, Encoder, CTC Head và bộ Detokenizer UTF-8 thành đồ thị tĩnh duy nhất chạy 100% NPU.
   - *Khối MT:* Tĩnh hóa kích thước ngữ cảnh và nhúng bảng tra cứu Language ID vào đồ thị QNN DLC của NLLB-200.
   - *Khối TTS:* Đóng gói liên hoàn Text Encoder, bộ tạo phổ Mel và Vocoder thành một chuỗi tính toán nội bộ duy nhất, xuất thẳng sóng âm PCM 16 kHz.
2. **Pha 2 – Ghép nối Toàn trình & Kiểm định Liên tầng (Inter-step Chaining):**

$$\text{Audio Vào} \xrightarrow{\text{Step 0: Lọc nhiễu}} \text{Step 1: ASR} \xrightarrow{\text{Step 2: Dịch MT}} \text{Step 3: Tổng hợp TTS} \rightarrow \text{Audio Ra}$$

   - *Truyền dữ liệu Zero-copy:* Các bước trao đổi dữ liệu bất đồng bộ qua bộ đệm tuần hoàn (*Circular Buffer*) trên RAM LPDDR5 theo mô hình Producer–Consumer, triệt tiêu độ trễ I/O.
   - *Cơ chế kiểm soát chất lượng liên tầng:* Tự động hạ chữ thường (`hyp.lower()`) từ Zipformer trước khi nạp vào MT để bảo toàn mã hóa BPE; đồng thời dùng SenseVoice nhận diện ngược âm thanh TTS để tự động sinh lại nếu phát hiện lỗi lặp âm tiết.
   - *Ngân sách độ trễ toàn trình (Latency Budget):* Tổng thời gian xử lý toàn chuỗi chỉ tốn 1.0–1.8 giây (ASR ~ 0.2 s, MT ~ 0.6 s, TTS ~ 0.5 s), hoàn toàn nằm dưới giới hạn khắt khe ≤ 3.0 giây của OneVoice AI Challenge.